# THub Web: guia para entender lo que estas construyendo

Apuntes personales de JSP, Servlets, MVC, capas, HTTP, Maven y seguridad




La idea central

Una pagina no suele hablar directamente con la base de datos. El navegador envia una peticion; el servidor la recibe, ejecuta el trabajo necesario y devuelve HTML o datos.

Preparado para estudiar con ejemplos de la estructura actual del proyecto THub. El proyecto sigue en desarrollo; esta guia describe el flujo que se observa ahora y distingue las ideas generales de los detalles concretos de la aplicacion.

# 1. El mapa general: MVC y capas

Tu aplicacion combina el patron MVC de la web con una arquitectura por capas. Son dos formas de mirar el mismo sistema: MVC ordena quien muestra, quien coordina y que datos usa; las capas separan responsabilidades tecnicas. No es necesario que cada carpeta sea una capa perfecta.




| Parte | Que hace | En THub |
|---|---|---|
| Vista / presentacion | Muestra la pagina y recoge lo que escribe la persona. | JSP, HTML, CSS y JavaScript bajo webapp. |
| Controlador | Recibe la request, interpreta la accion y coordina el siguiente paso. | Servlets como PublicEventServlet. |
| Negocio | Aplica reglas y organiza operaciones del dominio. | Servicios del middleware, por ejemplo EventService. |
| Acceso a datos | Habla con la base de datos y traduce filas a objetos. | DAO, JDBC y SQL del middleware. |
| Datos | Conserva la informacion de forma persistente. | Base de datos THub. |

Atajo para recordarlo
JSP presenta. Servlet controla. Service decide/coordina. DAO consulta o guarda. Base de datos conserva.

# 2. El viaje de una request

Una request es el mensaje que el navegador envia al servidor. En una busqueda, el mensaje lleva la URL, el metodo HTTP y los valores que la persona eligio en el formulario. Tomcat recibe la request y encuentra el servlet cuyo patron de URL coincide.




## Ejemplo actual: buscar eventos

- El boton del asidebar abre la vista publica de busqueda.
- La vista muestra criterios y listas. Al pulsar Buscar, el formulario envia sus campos y action=search a PublicEventServlet.
- El servlet convierte esos valores en un EventCriteria y llama a EventService.
- El servicio usa el DAO para consultar la base de datos; el resultado vuelve como objetos (por ejemplo, Results).
- El servlet guarda resultados/listas como atributos de request y hace forward a EventSearch.jsp.
- El JSP lee esos atributos y crea el HTML que Tomcat devuelve al navegador.

Dato util
Si no hay coincidencias, la request sigue siendo valida: la lista simplemente llega vacia. La JSP puede mostrar un mensaje y conservar los criterios.

# 3. Formularios, parametros y action

El atributo name de cada campo del formulario se convierte en el nombre del parametro HTTP. Por ejemplo, name="countryId" permite al servlet leer request.getParameter("countryId"). El nombre debe coincidir exactamente, incluidas mayusculas y minusculas.




## Que significa action

action es un parametro que tu aplicacion usa para que un mismo servlet haga tareas distintas. No es una palabra especial de Java ni de HTTP. En PublicEventServlet, por ejemplo, una accion puede mostrar el formulario y otra ejecutar la busqueda. El servlet mira su valor y entra en el bloque correspondiente.

## Flujo de los desplegables dependientes

En el buscador, cambiar el pais envia una request para volver a cargar la vista con las provincias relacionadas. Es una request completa y la pagina se vuelve a dibujar; el JavaScript no consulta por si mismo la base de datos. Luego la persona elige provincia y finalmente pulsa Buscar para obtener resultados. Asi se distingue "cargar opciones" de "buscar eventos".

> Request y session son distintas
> Los atributos de request viven durante una peticion y sirven para pasar datos del servlet a la JSP. La session puede mantener informacion entre peticiones, como el usuario que inicio sesion.

# 4. GET y POST, sin misterio

GET y POST son metodos HTTP: indican el tipo de operacion que solicita el navegador. La diferencia no es "seguro frente a inseguro" por si sola; HTTPS protege el transporte.

| | GET | POST |
|---|---|---|
| Donde van los datos | Normalmente en la URL, despues de ? (query string). | Normalmente en el cuerpo de la request. |
| Uso tipico | Consultar, buscar, abrir una pagina. | Enviar un formulario que crea o cambia algo. |
| Se puede guardar/enlazar | Si: la URL puede copiarse o marcarse. | Normalmente no: el envio no se representa como una URL reutilizable. |
| En THub | La busqueda de eventos es una consulta; puede viajar por GET. | Login, registro, cambios y subida de foto suelen usar POST. |

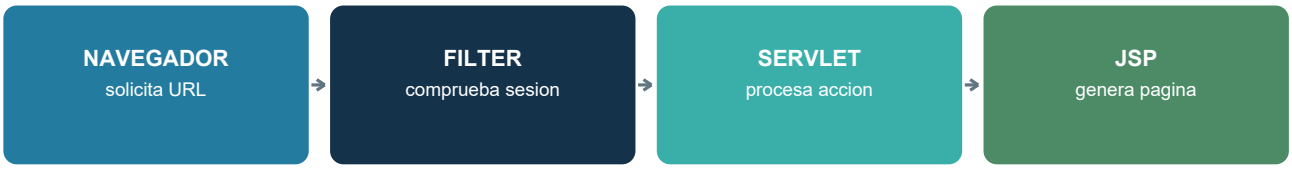
## Forward y redirect

- forward: el servidor entrega otra JSP dentro de la misma request. Los request attributes siguen disponibles; el navegador suele seguir viendo la URL del servlet.
- redirect: el servidor responde con una nueva direccion (normalmente 302) y el navegador hace otra request. Los request attributes anteriores no viajan; la session permanece.

Regla sencilla
GET suele leer. POST suele enviar cambios. Para borrar o modificar datos, conviene usar POST y validar siempre la operacion en el servidor.

# 5. JSP, Servlet y el AuthenticationFilter

Piensa en estas piezas como roles diferentes. El servlet es el mostrador que recibe la solicitud. El filtro es el control de acceso que puede dejarla pasar o enviarla al login antes de llegar al mostrador. La JSP es la plantilla con la que el servidor prepara la pagina.




## Como trabaja el filtro en este proyecto

- AuthenticationFilter obtiene de la session el usuario bajo la clave centralizada usuario.
- Si encuentra un usuario, llama a chain.doFilter(...). Eso significa: continua con el siguiente componente de la cadena.
- Si no hay usuario, redirige a la vista de login y no deja continuar con la peticion protegida.
- web.xml registra la clase y sus patrones actuales cubren /Private/* y /html/private/*.

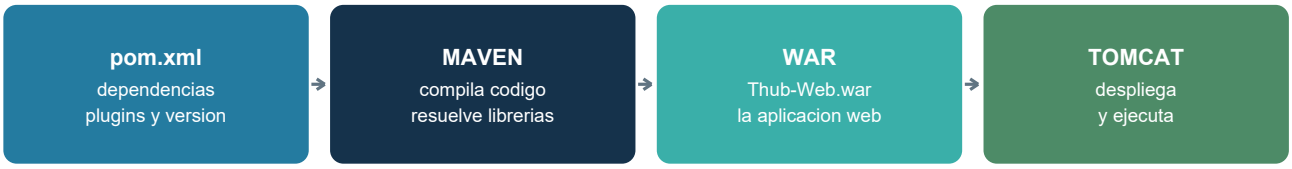
## Que pinta web.xml

web.xml es el descriptor de despliegue web. Tomcat lo lee al iniciar la aplicacion. En este proyecto declara el filtro, donde se aplica y la pagina inicial (index.jsp). Una ruta no protegida por el mapping del filtro no pasa por esa comprobacion. Las rutas distinguen mayusculas y minusculas.

Autenticacion y permisos
El filtro responde "hay alguien conectado?". Una regla de permisos responde "esta persona puede editar este evento concreto?". Para permitir editar solo eventos propios, el servlet/service tambien debe comprobar la propiedad del evento.

# 6. Maven: construir y organizar dependencias

Maven es una herramienta de construccion para proyectos Java. Lee el archivo pom.xml, descarga/resuelve dependencias y ejecuta tareas estandar para compilar y empaquetar. El pom es como la ficha de instrucciones del proyecto: identidad, librerias, versiones y plugins.




## Ejemplos que aparecen en el pom de THub

- Java 21: version para compilar el codigo fuente.
- Jakarta Servlet API: permite compilar contra la API de servlets. Va con scope provided porque Tomcat aporta esa API al ejecutar la aplicacion.
- thub-middleware: dependencia del otro proyecto que contiene servicios, DAO y objetos compartidos.
- Commons FileUpload: ayuda a interpretar formularios multipart que incluyen archivos.
- Packaging WAR: Maven empaqueta la aplicacion web en un archivo que Tomcat puede desplegar.

## Ciclo practico

mvn clean package limpia artefactos anteriores, compila y prepara el paquete. El directorio target contiene resultados generados. En Eclipse, compilar el WAR y publicarlo en Tomcat son pasos relacionados pero distintos: el servidor debe recibir las dependencias correctas en WEB-INF/lib.

Por que "provided" importa
Una dependencia con scope provided se usa para compilar, pero Maven no la incluye como libreria normal dentro del WAR. Es apropiado para Servlet API porque Tomcat ya la proporciona. No se debe aplicar sin pensar a una libreria propia necesaria en ejecucion.

# 7. Sesion, subida de foto y contexto de la aplicacion

Al iniciar sesion, el servidor puede guardar el objeto usuario en la session. En cada request posterior, el filtro/servlet recupera el mismo usuario. Asi no hay que volver a pedir el nombre y la contrasena en cada pagina. La clave de session en THub se centraliza como usuario.




## Flujo simplificado de foto de perfil

- El formulario de subida usa enctype="multipart/form-data" porque envia bytes de archivo junto con los demas campos.
- El servlet identifica la accion, obtiene los bytes con la utilidad de lectura multipart y valida que haya archivo.
- El usuario de la session aporta el ID confiable para guardar el contenido asociado a esa cuenta.
- El servicio CMS del middleware guarda los bytes. Otra accion puede devolver la imagen con su tipo MIME (por ejemplo image/png).

## Context path

El context path es el prefijo con que Tomcat publica la aplicacion, por ejemplo /Thub-Web. Las JSP suelen construir las direcciones con request.getContextPath() para que funcionen aunque la aplicacion tenga otro nombre de despliegue.

> Cuidado con IDs enviados por el navegador
> Un campo oculto sigue siendo modificable. Para operaciones privadas, el ID de usuario debe salir de la session; el servidor no debe confiar en un ID recibido desde el formulario.

# 8. Mini glosario y como investigar un fallo

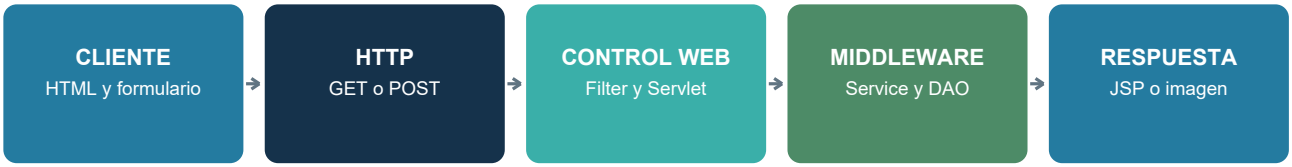
| Termino | En palabras sencillas |
|---|---|
| Request | Peticion HTTP del navegador al servidor. |
| Response | Respuesta HTTP del servidor al navegador. |
| Parametro | Dato enviado en la URL o en el cuerpo del formulario; se lee por su nombre. |
| Atributo de request | Dato temporal que el servlet entrega a la JSP en esa peticion. |
| Session | Espacio de datos asociado a una persona/navegador entre varias requests. |
| DTO | Objeto sencillo para transportar datos entre partes del programa. |
| Criteria | Objeto que agrupa filtros de busqueda. |
| DAO | Componente que consulta o cambia datos en la base de datos. |
| JSP | Plantilla procesada en el servidor para producir HTML. |
| WAR | Paquete de aplicacion web que se despliega en Tomcat. |

## Cuando algo no funciona, sigue el viaje

- [ ] Que URL exacta se solicito? Incluye context path y action?
- [ ] El formulario usa el metodo esperado y los name coinciden con getParameter?
- [ ] Tomcat mapea esa URL al servlet correcto? Hay un filtro en esa ruta?
- [ ] El servlet recibe valores? Comprueba los parametros y los logs.
- [ ] El servicio encuentra/lanza un error? El DAO y la consulta devuelven datos?
- [ ] La JSP recibe los atributos con los mismos nombres que intenta mostrar?
- [ ] Tras cambiar JSP/CSS, se guardo y se recargo la pagina publicada?

Orden de diagnostico
Empieza por el navegador y avanza hacia el servidor: URL/formulario -> filtro -> servlet -> servicio -> DAO/base de datos -> atributos -> JSP. Asi encuentras en que tramo se pierde el dato.

# 9. Resumen para tener a mano




## Si solo recuerdas siete cosas

- El navegador solo conoce URLs y envia requests; no llama directamente a Java ni a la base de datos.
- El filtro revisa peticiones protegidas antes del servlet.
- El servlet interpreta la accion y coordina el trabajo.
- El service contiene/organiza la logica; el DAO habla con la base de datos.
- El servlet pasa datos a la JSP usando request attributes; la JSP dibuja la respuesta.
- GET suele consultar; POST suele enviar cambios. El servidor siempre valida los datos.
- Maven compila, organiza dependencias y empaqueta; Tomcat ejecuta el WAR.

Tu ejemplo mental para buscar eventos
"Elijo filtros y pulso Buscar. El navegador envia una request; el filtro decide si la ruta necesita login; PublicEventServlet lee los parametros, crea los criterios y llama al servicio; el middleware consulta la base de datos; el servlet entrega los resultados a EventSearch.jsp; Tomcat devuelve HTML y yo veo la lista."

Estos apuntes describen el codigo observado en el proyecto el 8 de noviembre de 2026. Algunas rutas y funciones siguen en construccion; cuando evolucione el codigo, actualiza los ejemplos para que coincidan con la implementacion.